L’algorithme de Little

Michaël Soulignac

ESILV Nantes

Principe de l’algorithme de Little

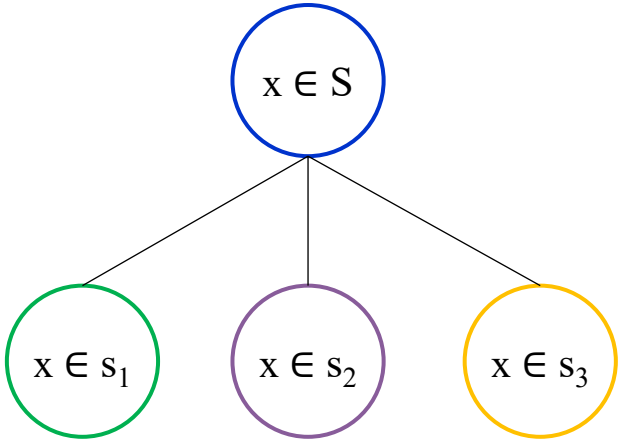
L'algorithme de Little

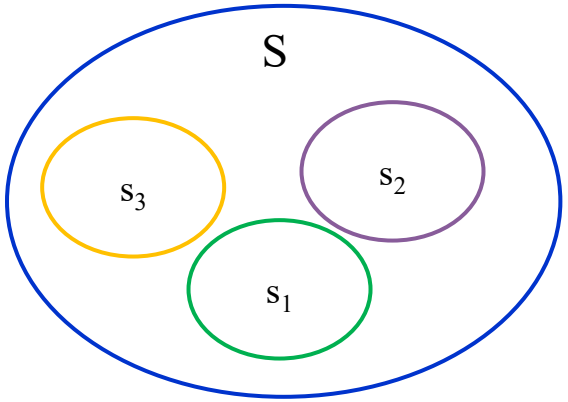
- Algorithme de résolution exacte du problème du voyageur de commerce (*Traveling Salesman Problem – TSP*)
- Introduit en 1963 par Little *et al*
- Elague fortement l'arbre de recherche en utilisant une stratégie de séparation & évaluation (*branch & bound*)

Séparation & évaluation (*branch & bound*)

1. **Branch** (séparation) : on sépare le problème initial en sous-problèmes, en considérant des sous-ensembles de l'espace des solutions

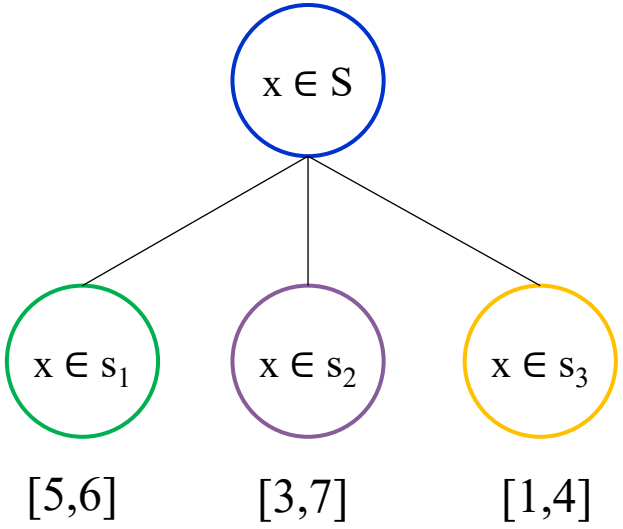
$\Rightarrow$ on obtient un arbre dans lequel chaque nœud modélise un sous-problème

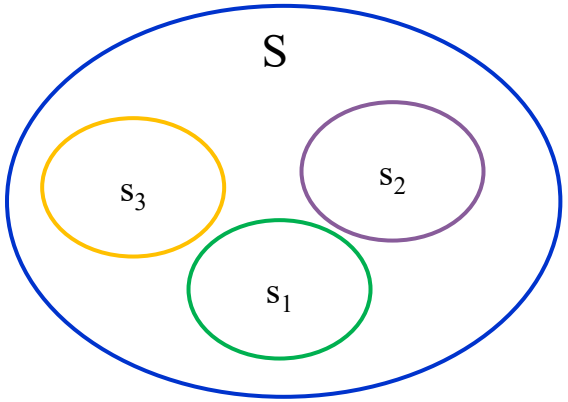




Séparation & évaluation (*branch & bound*)

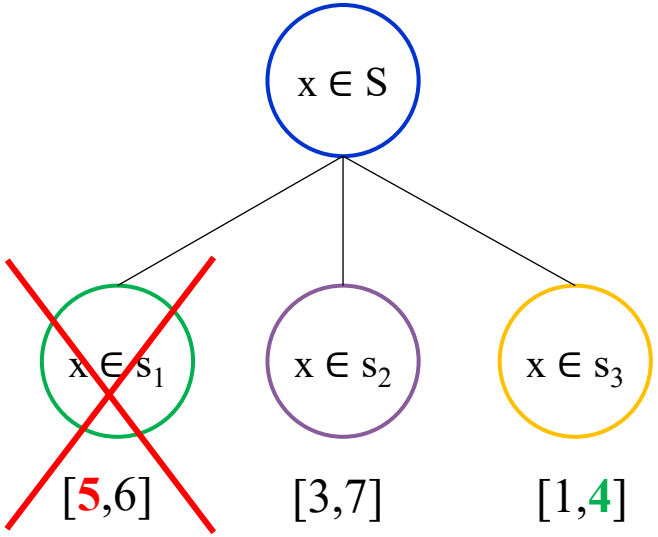
2. **Bound** (évaluation) : pour chaque nœud, on évalue un intervalle pour le coût des solutions encore réalisables

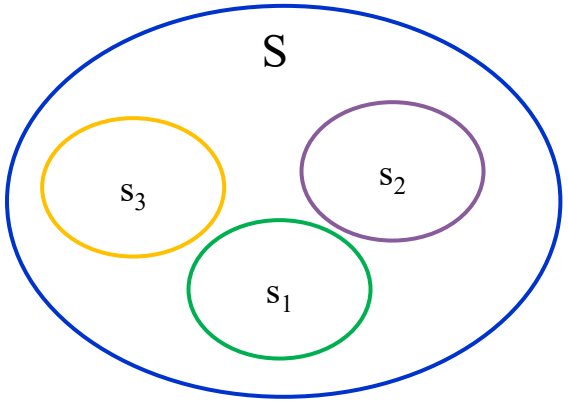




Séparation & évaluation (*branch & bound*)

3. **Cut** (coupure) : si un nœud N a été évalué à [a,b] et s'il existe un autre nœud N' évalué à [c,d] tel que $a > d$, alors on peut couper la branche au niveau de N sans risque de manquer la solution optimale



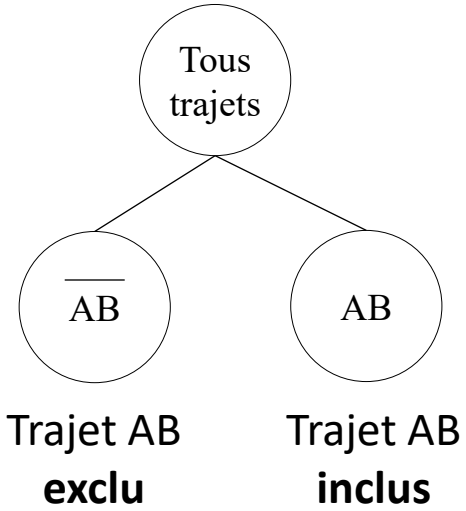


Le *Branch & bound* dans l'algorithme de Little

1. **Branch** : Inclusion ou exclusion d'un trajet dans les tournées encore possibles

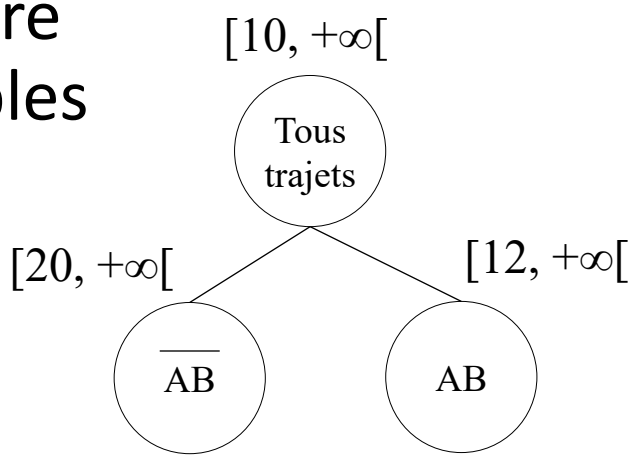
$\Rightarrow$ Arbre de recherche binaire

La racine considère que tous les trajets sont utilisables



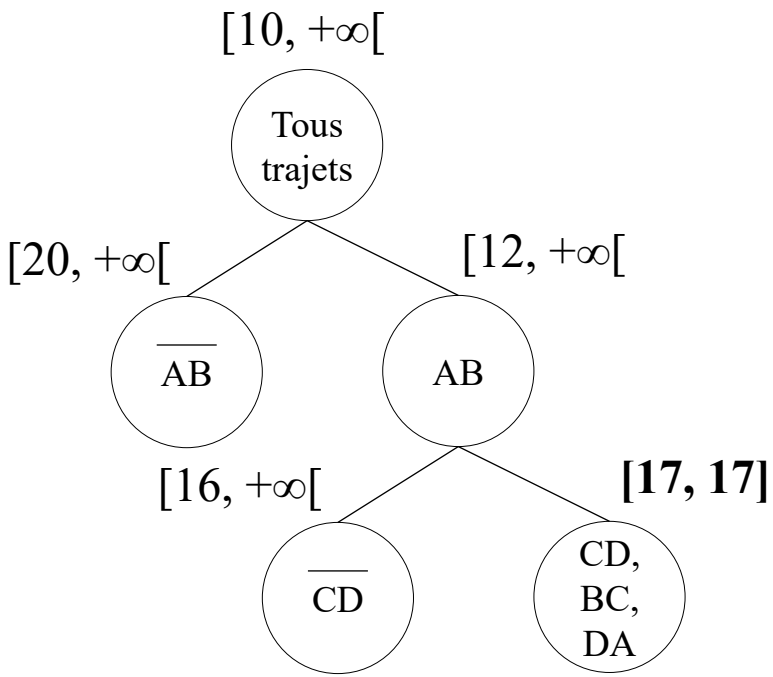
Le *Branch & bound* dans l'algorithme de Little

2. **Bound** : Evaluation d'une borne inférieure pour le coût des tournées encore possibles



Le *Branch & bound* dans l'algorithme de Little

On développe l'arbre de recherche à droite d'abord (trajets inclus) pour obtenir une première solution, et donc une borne supérieure finie



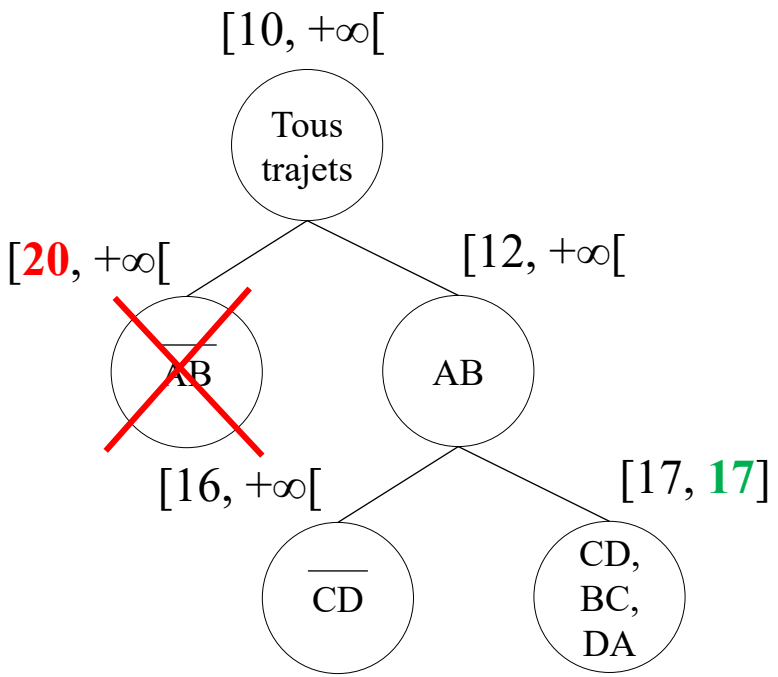
En effet, une fois une première tournée identifiée, on a :

borne inférieure = borne supérieure = coût de la tournée

Le *Branch & bound* dans l'algorithme de Little

3. **Cut** : On tente d'améliorer cette première tournée en explorant les branches restantes

⇒ On coupe toutes les branches ayant nécessairement un coût supérieur (ou égal)

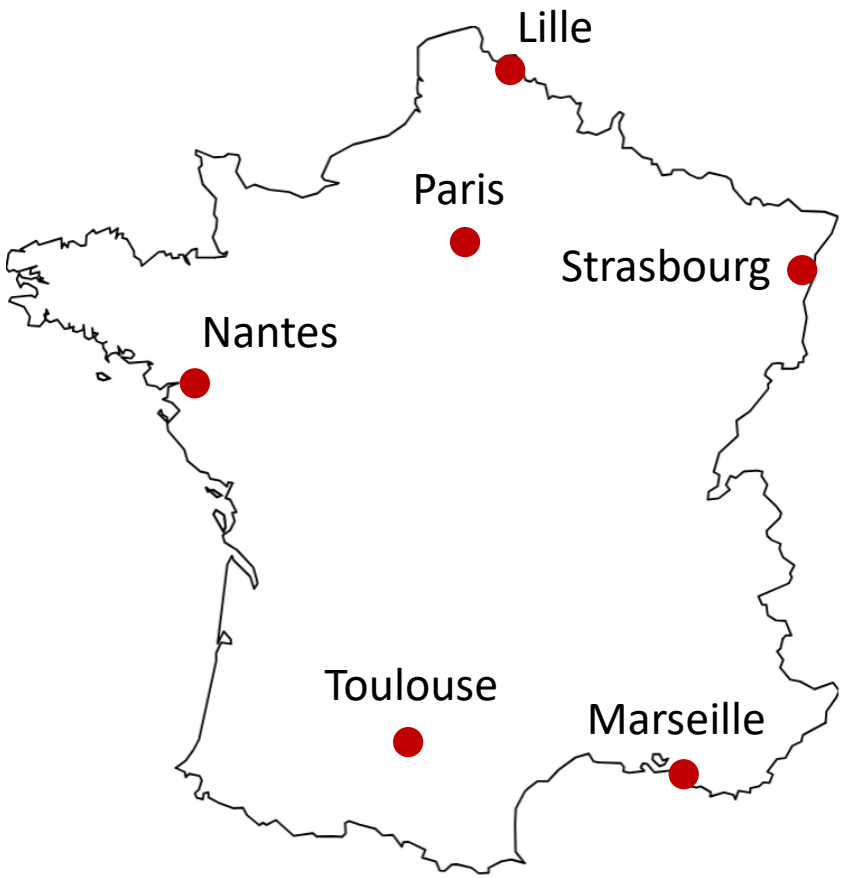


Exemple

Exemple

- 6 villes françaises : permet d'illustrer les trajets sur la carte
- Coûts symétriques ici, mais pas en général

⇒ Un exemple avec des coûts asymétriques sera étudié en TD



Evaluation d'un nœud

- La valeur d'un nœud représente la borne inférieure du coût des solutions encore possibles
- Cette valeur ne correspond pas forcément à coût atteignable : on sait juste que les tournées restantes ont **au moins** ce coût
- Elle est calculée en effectuant une opération sur la matrice des coûts appelée **réduction**.

Réduction de la matrice des coûts

Matrice des coûts = distances en km, à vol d'oiseau

Coûts diagonaux infinis

	N	P	T	L	M	S
N	+∞	343	465	507	697	710
P	343	+∞	588	210	661	397
T	465	588	+∞	791	407	737
L	507	210	791	+∞	834	397
M	697	661	407	834	+∞	615
S	710	397	737	397	615	+∞

Réduction de la matrice des coûts

1. Soustraction du minimum ligne par ligne

(Soustraire un nombre quelconque à une ligne ne change pas le problème car cela revient à soustraire cette distance à toutes les tournées)

	N	P	T	L	M	S	*min*
N	+∞	343	465	507	697	710	*343*
P	343	+∞	588	210	661	397	*210*
T	465	588	+∞	791	407	737	*407*
L	507	210	791	+∞	834	397	*210*
M	697	661	407	834	+∞	615	*407*
S	710	397	737	397	615	+∞	*397*

⇨

	N	P	T	L	M	S
N	+∞	0	122	164	354	367
P	133	+∞	378	0	451	187
T	58	181	+∞	384	0	330
L	297	0	581	+∞	624	187
M	290	254	0	427	+∞	208
S	313	0	340	0	218	+∞

Réduction des lignes : 1974 ⇒ coût minimal pour partir de chaque ville

Réduction de la matrice des coûts

1. Soustraction du minimum ligne par ligne
2. Soustraction du minimum colonne par colonne

	N	P	T	L	M	S	*min*
N	+∞	343	465	507	697	710	*343*
P	343	+∞	588	210	661	397	*210*
T	465	588	+∞	791	407	737	*407*
L	507	210	791	+∞	834	397	*210*
M	697	661	407	834	+∞	615	*407*
S	710	397	737	397	615	+∞	*397*

	N	P	T	L	M	S
N	+∞	0	122	164	354	367
P	133	+∞	378	0	451	187
T	58	181	+∞	384	0	330
L	297	0	581	+∞	624	187
M	290	254	0	427	+∞	208
S	313	0	340	0	218	+∞
min	*58*	*0*	*0*	*0*	*58*	*187*

Matrice réduite

	N	P	T	L	M	S
N	+∞	0	122	164	354	180
P	75	+∞	378	0	451	0
T	0	181	+∞	384	0	143
L	239	0	581	+∞	624	0
M	232	254	0	427	+∞	21
S	255	0	340	0	218	+∞

Réduction des lignes : 1974 **+** Réduction des colonnes : 245

Réduction de la matrice des coûts

1. Soustraction du minimum ligne par ligne
2. Soustraction du minimum colonne par colonne
3. Valeur du nœud = réduction totale

	N	P	T	L	M	S	*min*
N	+∞	343	465	507	697	710	*343*
P	343	+∞	588	210	661	397	*210*
T	465	588	+∞	791	407	737	*407*
L	507	210	791	+∞	834	397	*210*
M	697	661	407	834	+∞	615	*407*
S	710	397	737	397	615	+∞	*397*

	N	P	T	L	M	S
N	+∞	0	122	164	354	367
P	133	+∞	378	0	451	187
T	58	181	+∞	384	0	330
L	297	0	581	+∞	624	187
M	290	254	0	427	+∞	208
S	313	0	340	0	218	+∞
min	*58*	*0*	*0*	*0*	*58*	*187*

Matrice réduite

	N	P	T	L	M	S
N	+∞	0	122	164	354	180
P	75	+∞	378	0	451	0
T	0	181	+∞	384	0	143
L	239	0	581	+∞	624	0
M	232	254	0	427	+∞	21
S	255	0	340	0	218	+∞

Réduction des lignes : 1974 **+** Réduction des colonnes : 245 **=** **Réduction totale : 2219**

Notion de regret

Pour choisir sur quel trajet on va effectuer la prochaine séparation, on introduit la notion de **regret** :

- Associé à un trajet de valeur 0 dans la matrice réduite
- Représente le **surcoût minimal** induit par l'exclusion de ce trajet

Calcul d'un regret

Regret de MT = 21 + 122 = 143

	N	P	T	L	M	S	
N	+∞	0	122	164	354	180	
P	75	+∞	378	0	451	0	
T	0	181	+∞	384	0	143	
L	239	0	581	+∞	624	0	*min ligne :*
M	232	254	0 **(143)**	427	+∞	21	*21*
S	255	0	340	0	218	+∞	
			min colonne : 122				

Regret = somme des coûts minimaux des trajets alternatifs pour :

- Partir de la ville source = min de la ligne*
- Arriver à la ville destination = min de la colonne*

* Sans le trajet exclu, évidemment !

Calcul des regrets

	N	P	T	L	M	S
N	+∞	0 **(122)**	122	164	354	180
P	75	+∞	378	0 **(0)**	451	0 **(0)**
T	0 **(75)**	181	+∞	384	0 **(218)**	143
L	239	0 **(0)**	581	+∞	624	0 **(0)**
M	232	254	0 **(143)**	427	+∞	21
S	255	0 **(0)**	340	0 **(0)**	218	+∞

Heuristique de séparation

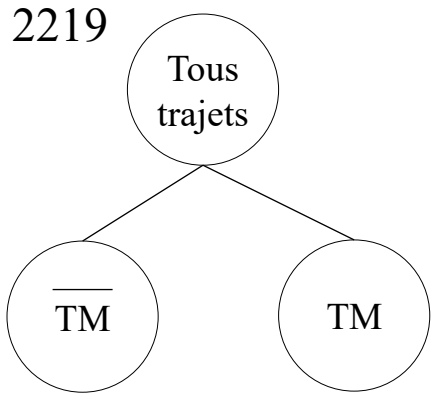
	N	P	T	L	M	S
N	+∞	0 **(122)**	122	164	354	180
P	75	+∞	378	0 **(0)**	451	0 **(0)**
T	0 **(75)**	181	+∞	384	0 **(218)**	143
L	239	0 **(0)**	581	+∞	624	0 **(0)**
M	232	254	0 **(143)**	427	+∞	21
S	255	0 **(0)**	340	0 **(0)**	218	+∞

On choisit le trajet de **regret maximal** *(ici Toulouse-Marseille – TM)*

⇒ rend la borne inférieure la plus discriminante possible

⇒ maximise les chances d'élaguer les branches rapidement

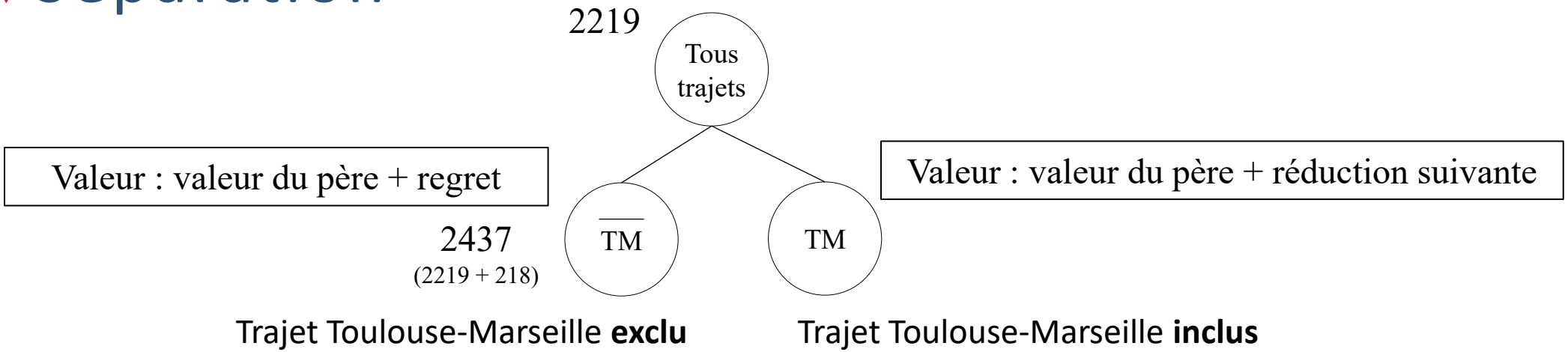
Séparation



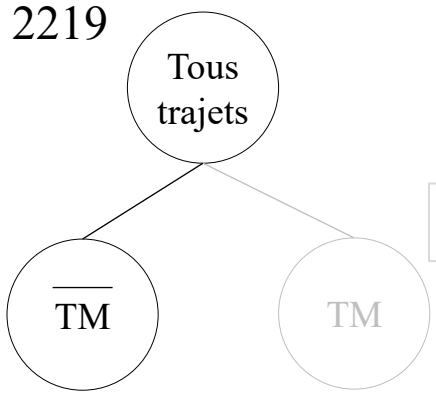
Trajet Toulouse-Marseille **exclu**

Trajet Toulouse-Marseille **inclus**

Séparation


2219
Tous trajets
Valeur : valeur du père + regret
Valeur : valeur du père + réduction suivante
2437
(2219 + 218)
$\overline{\text{TM}}$
TM
Trajet Toulouse-Marseille **exclu**
Trajet Toulouse-Marseille **inclus**

Matrices des nœuds suivants



Valeur : valeur du père + regret

2437
(2219 + 218)

Valeur : valeur du père + réduction suivante

Trajet Toulouse-Marseille **exclu**

Trajet Toulouse-Marseille **inclus**

	N	P	T	L	M	S
N	+∞	0	122	164	354	180
P	75	+∞	378	0	451	0
T	0	181	+∞	384	+∞	143
L	239	0	581	+∞	624	0
M	232	254	0	427	+∞	21
S	255	0	340	0	218	+∞

Suppression de la case correspondant au trajet exclu

La réduction de cette matrice donnerait :

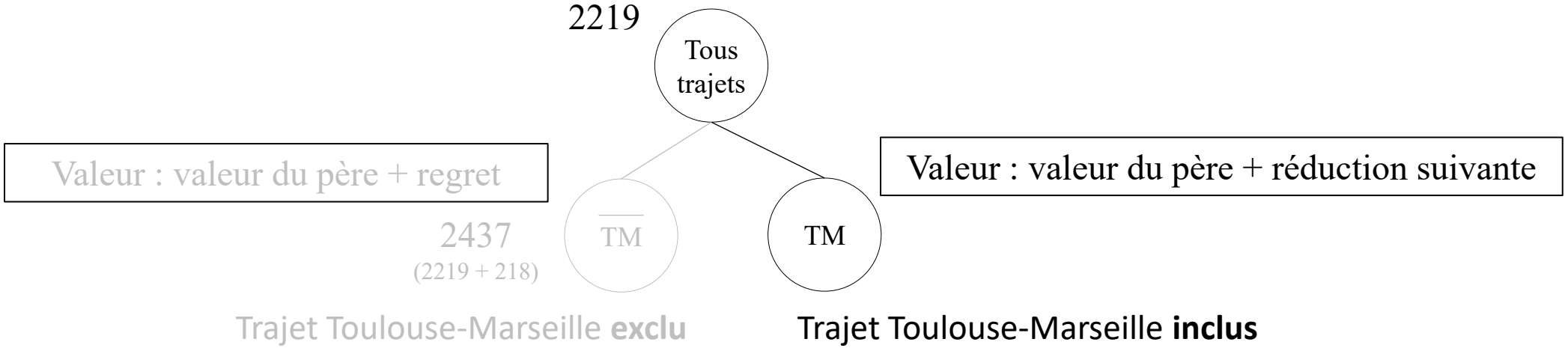
0 (lignes) + 218 (colonnes) = 218

ce qui est égal au regret, par définition

⇒ On connaît la valeur du nœud **avant** de le visiter

⇒ Ceci permet de couper des branches prématurément

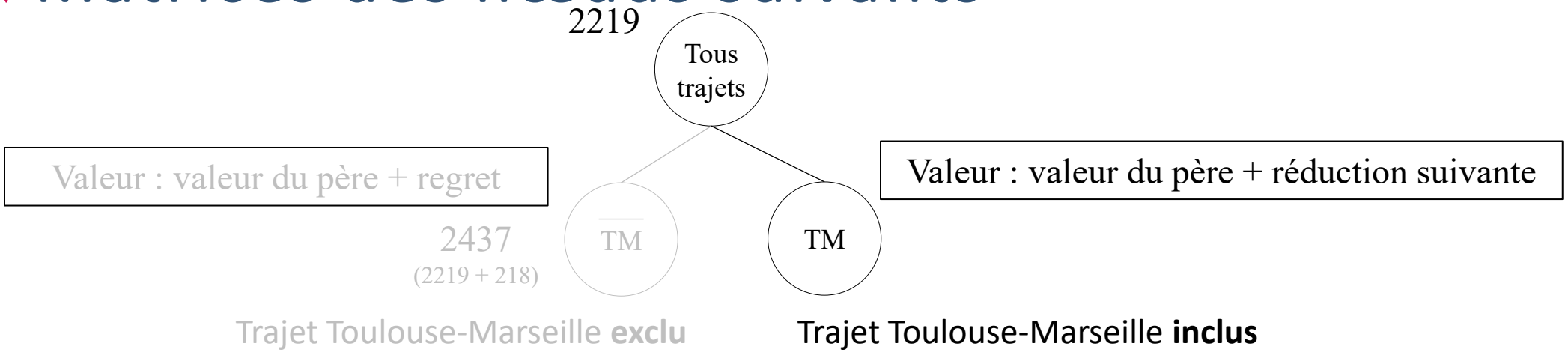
Matrices des nœuds suivants



1. Suppression de la ligne et de la colonne du trajet inclus (*ici ligne T et colonne M*)

	N	P	T	L	S
N	+∞	0	122	164	180
P	75	+∞	378	0	0
L	239	0	581	+∞	0
M	232	254	0	427	21
S	255	0	340	0	+∞

Matrices des nœuds suivants



1. Suppression de la ligne et de la colonne du trajet inclus (*ici ligne T et colonne M*)
2. Suppression des trajets dits « parasites », amenant à des tournées incomplètes (*ici MT ⇒ tournée TM-MT*)

	N	P	T	L	S
N	+∞	0	122	164	180
P	75	+∞	378	0	0
L	239	0	581	+∞	0
M	232	254	+∞	427	21
S	255	0	340	0	+∞

⚠ Les trajets parasites ne sont pas simplement l'inverse des trajets inclus. **Il faut détecter des cycles !**

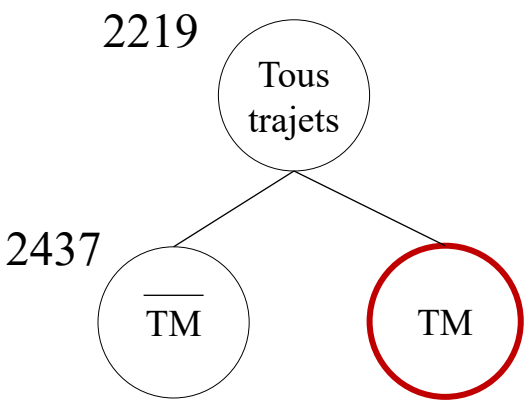
Récursion

On procède ensuite de manière récursive :

1. Appel récursif de droite : trajet inclus
2. Appel récursif de gauche : trajet exclu

⇒ Ceci va naturellement développer un arbre de recherche à droite d'abord, et aboutir à une première solution, en mettant les nœuds de type « trajet exclu » en attente (dans la pile d'exécution)

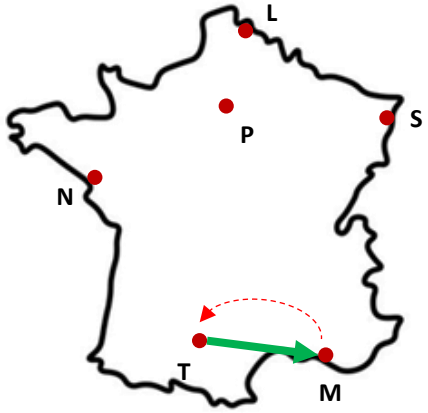
Appel récursif n°1 – droite



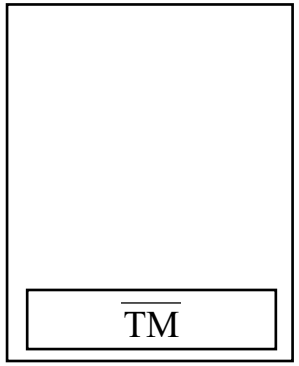
	N	P	T	L	S
N	+∞	0	122	164	180
P	75	+∞	378	0	0
L	239	0	581	+∞	0
M	232	254	+∞	427	21
S	255	0	340	0	+∞

Trajet parasite éliminé : MT

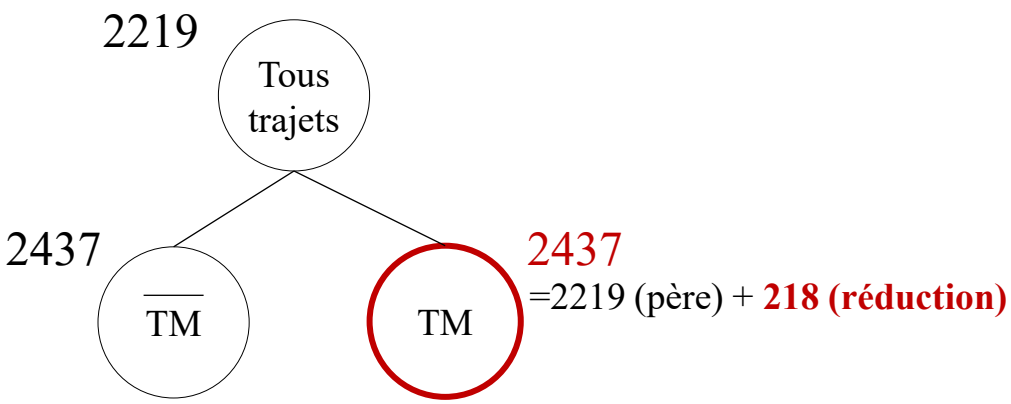
⇒ sous-tournée TM-MT



Pile d'exécution



Appel récursif n°1 – droite

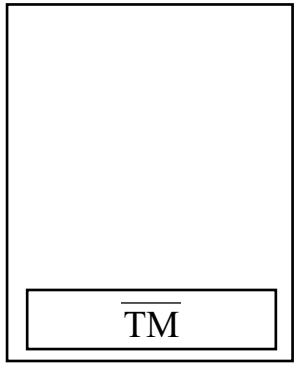


	N	P	T	L	S
N	+∞	0	122	164	180
P	75	+∞	378	0	0
L	239	0	581	+∞	0
M	232	254	+∞	427	21
S	255	0	340	0	+∞

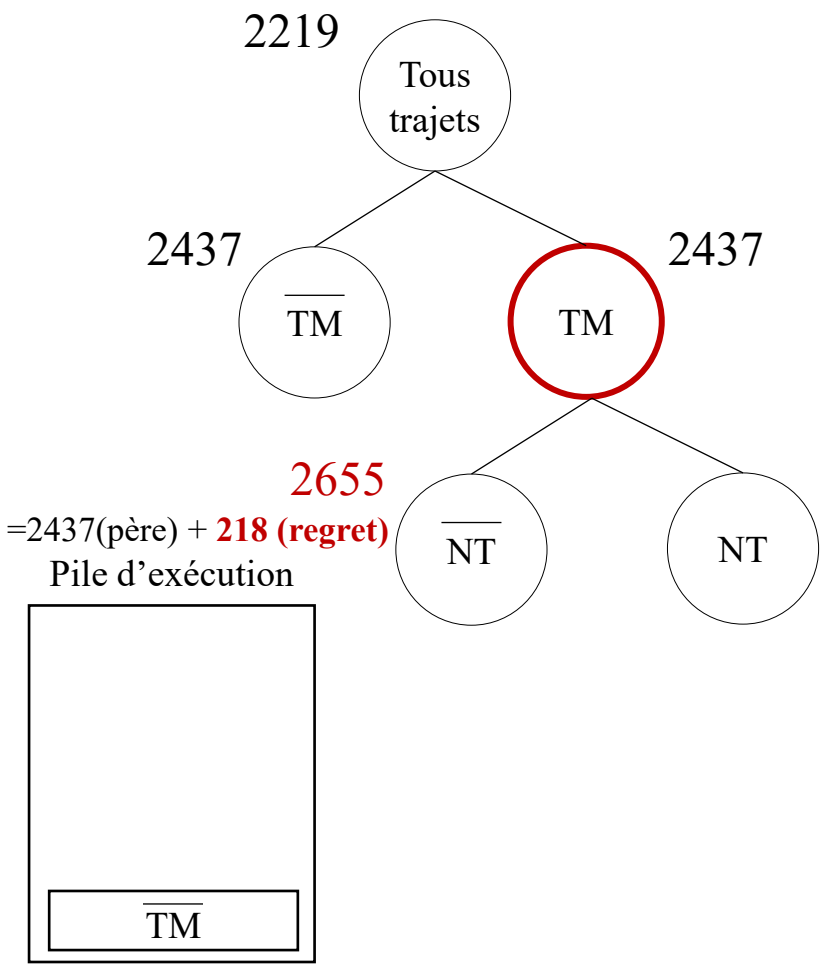
Réduction : 218
= 21 (lignes) + 197 (colonnes)

	N	P	T	L	S
N	+∞	0	0	164	180
P	0	+∞	378	0	0
L	164	0	459	+∞	0
M	136	233	+∞	406	0
S	180	0	218	0	+∞

Pile d'exécution



Appel récursif n°1 – droite



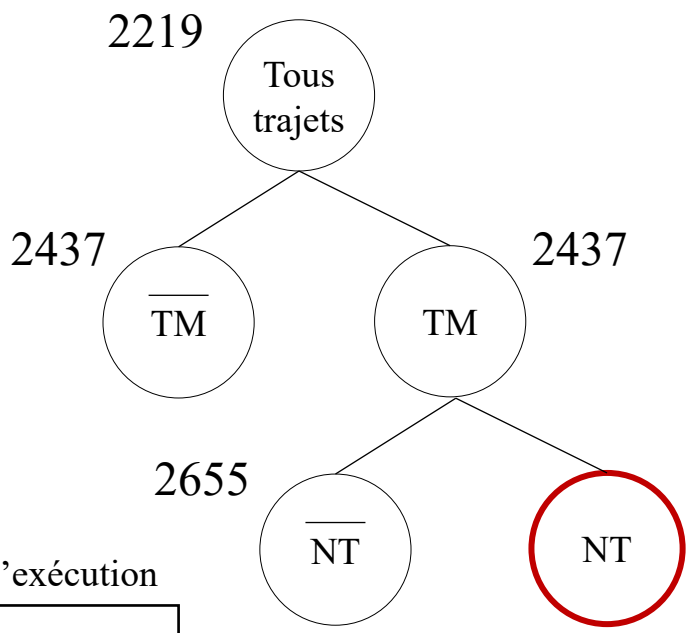
	N	P	T	L	S
N	+∞	0	122	164	180
P	75	+∞	378	0	0
L	239	0	581	+∞	0
M	232	254	+∞	427	21
S	255	0	340	0	+∞

Réduction

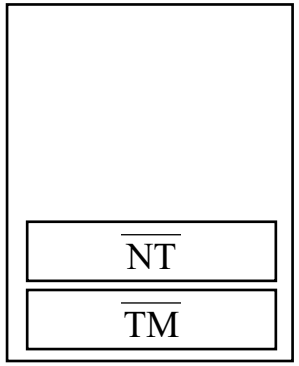
	N	P	T	L	S
N	+∞	0 **(0)**	0 **(218)**	164	180
P	0 **(136)**	+∞	378	0 **(0)**	0 **(0)**
L	164	0 **(0)**	459	+∞	0 **(0)**
M	136	233	+∞	406	0 **(136)**
S	180	0 **(0)**	218	0 **(0)**	+∞

Regret maximal : Nantes-Toulouse

Appel récursif n°2 – droite



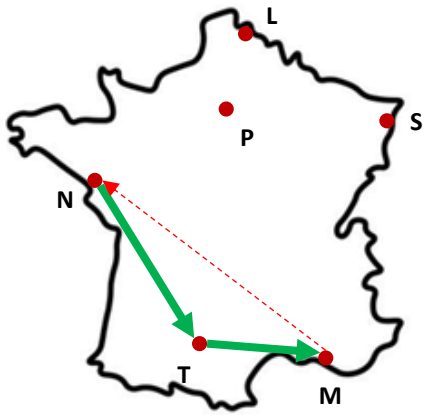
Pile d'exécution



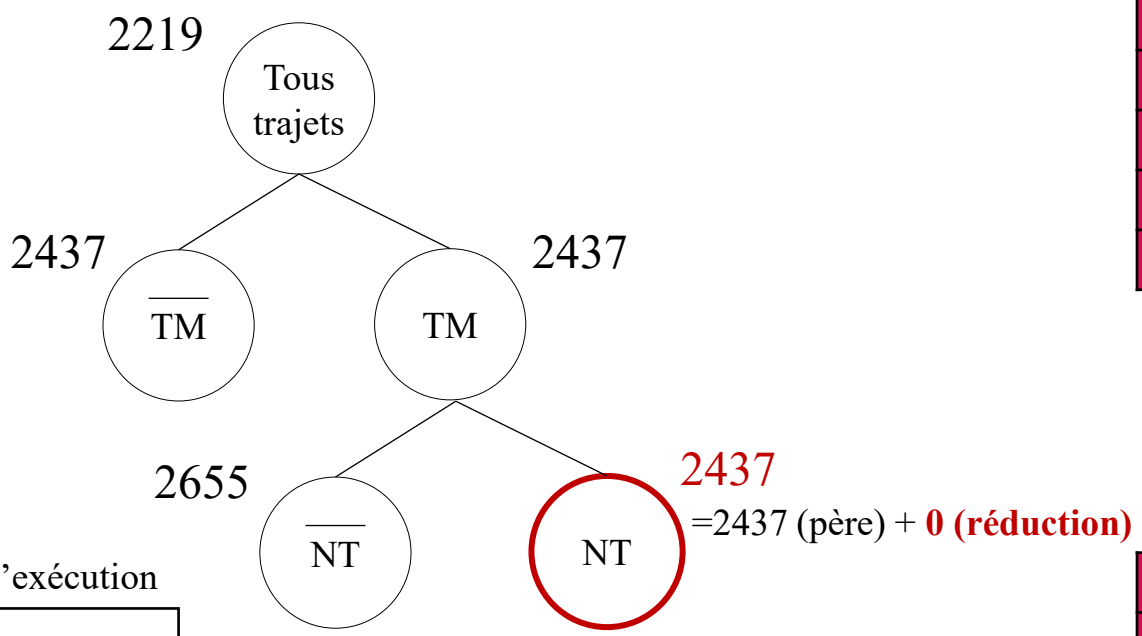
	N	P	L	S
P	0	+∞	0	0
L	164	0	+∞	0
M	+∞	233	406	0
S	180	0	0	+∞

Trajet parasite éliminé : MN

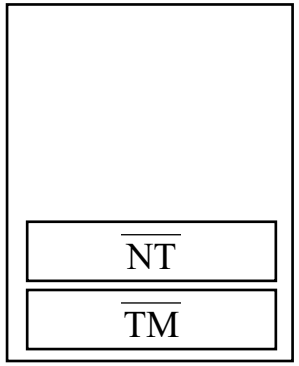
⇒ sous-tournée NT-TM-MN



Appel récursif n°2 – droite



Pile d'exécution

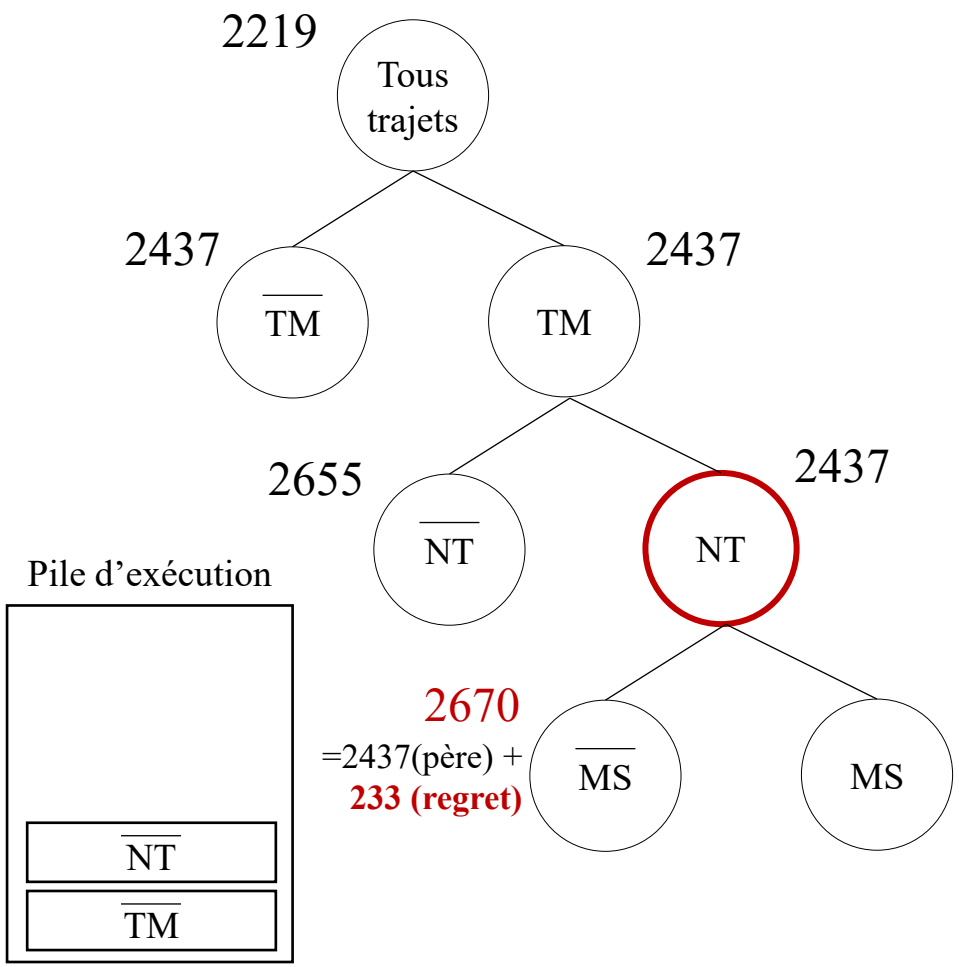


	N	P	L	S
P	0	+∞	0	0
L	164	0	+∞	0
M	+∞	233	406	0
S	180	0	0	+∞

Réduction : 0

	N	P	L	S
P	0	+∞	0	0
L	164	0	+∞	0
M	+∞	233	406	0
S	180	0	0	+∞

Appel récursif n°2 – droite



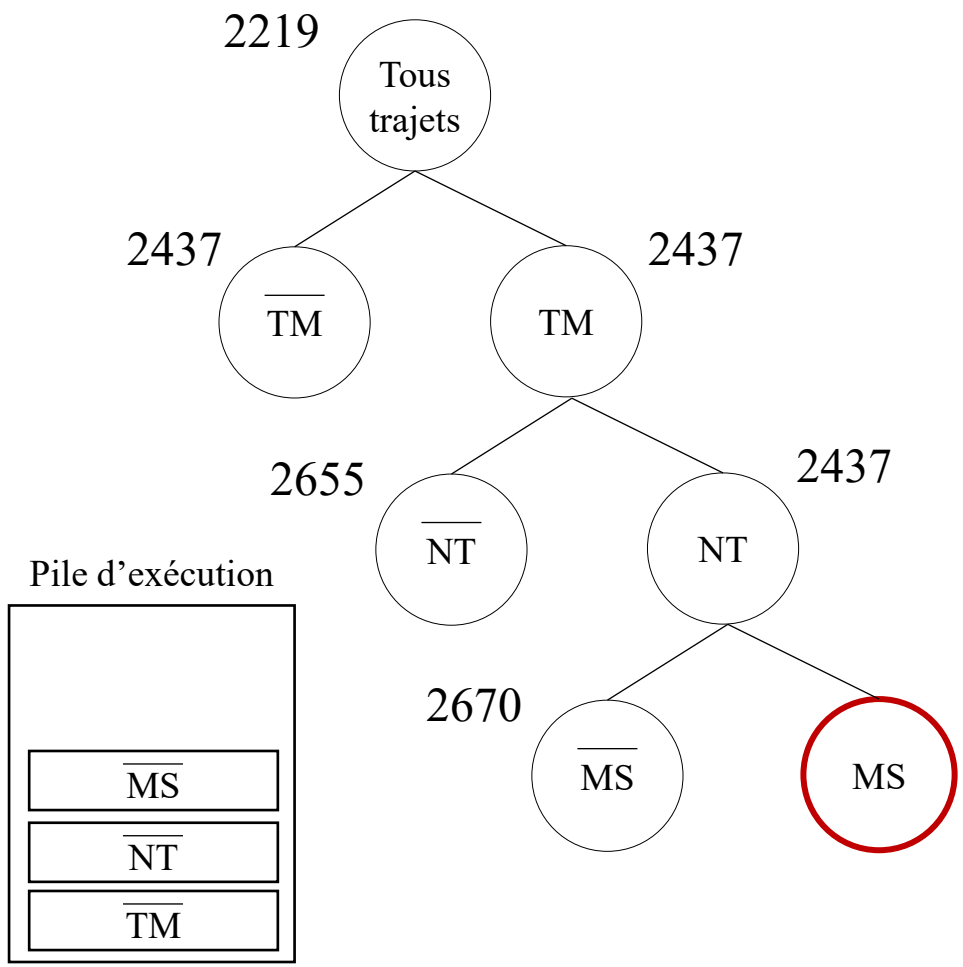
	N	P	L	S
P	0	+∞	0	0
L	164	0	+∞	0
M	+∞	233	406	0
S	180	0	0	+∞

Réduction : 0

	N	P	L	S
P	0 **(164)**	+∞	0 **(0)**	0 **(0)**
L	164	0 **(0)**	+∞	0 **(0)**
M	+∞	233	406	0 **(233)**
S	180	0 **(0)**	0 **(0)**	+∞

Regret maximal : Marseille-Strasbourg

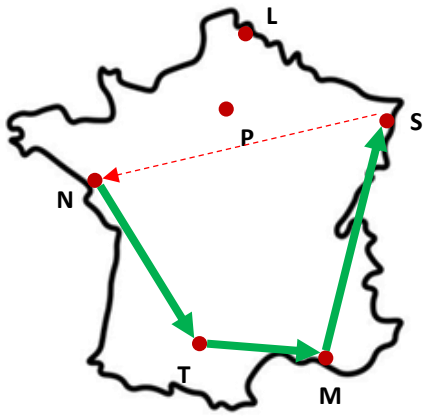
Appel récursif n°3 – droite



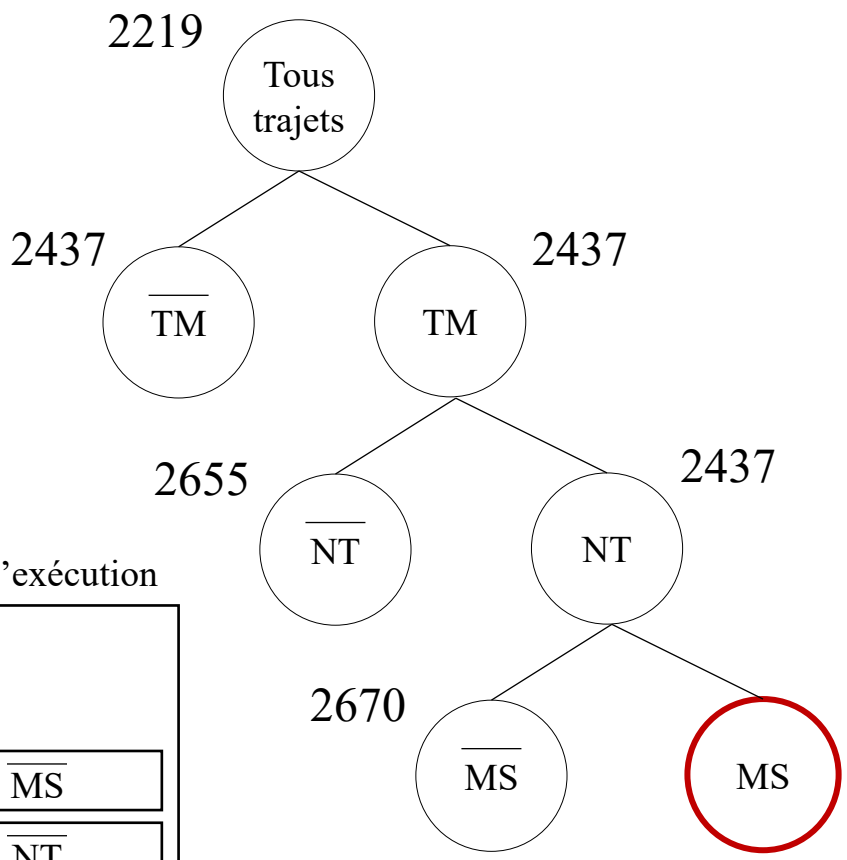
	N	P	L
P	0	+∞	0
L	164	0	+∞
S	+∞	0	0

Trajet parasite éliminé : SN

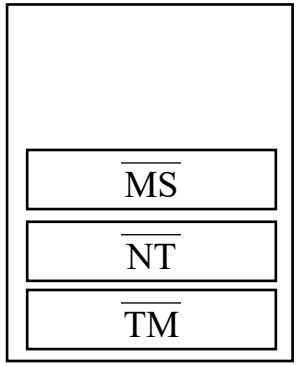
⇒ sous-tournée NT-TM-MS-SN



Appel récursif n°3 – droite



Pile d'exécution

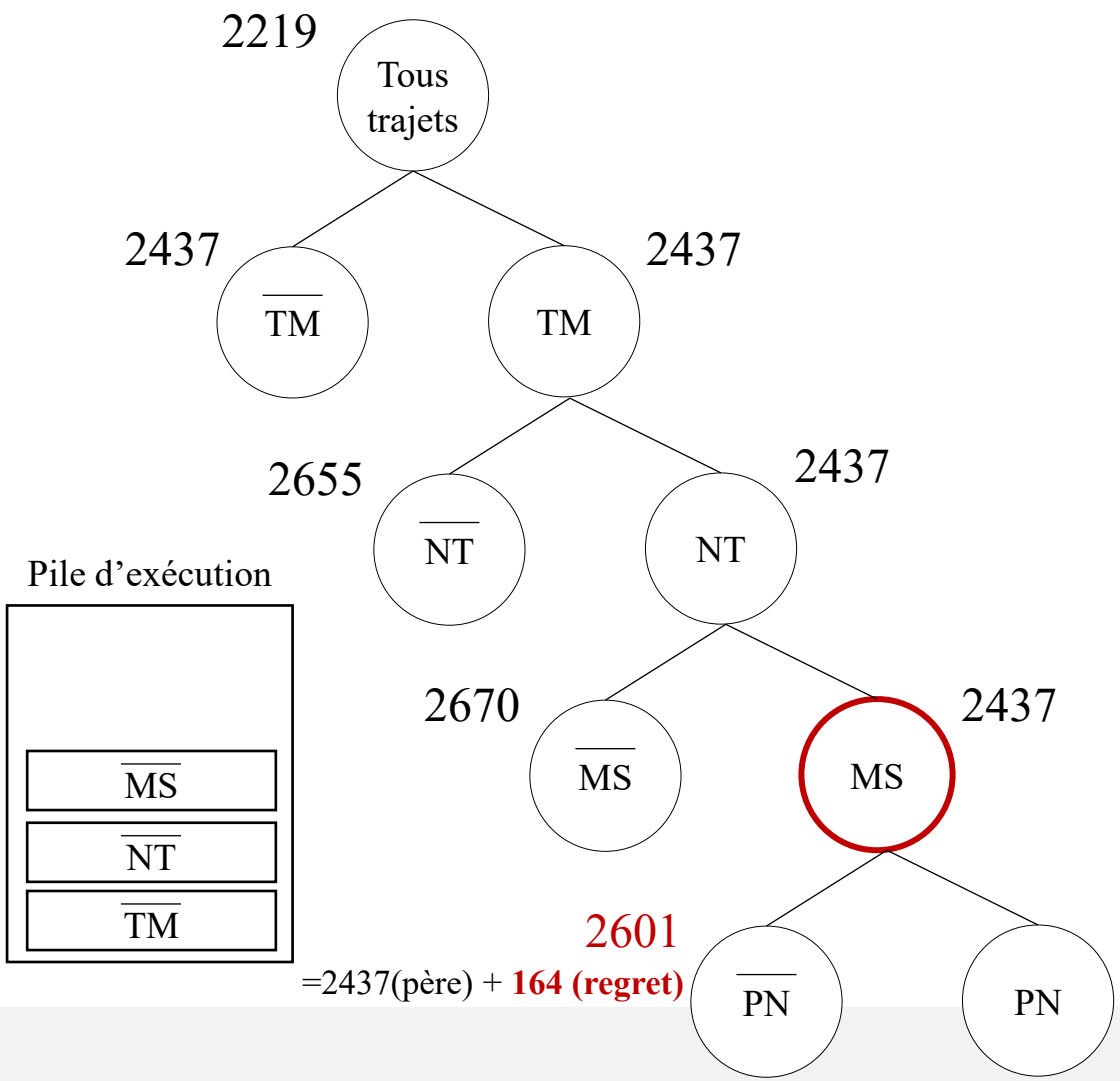


	N	P	L
P	0	+∞	0
L	164	0	+∞
S	+∞	0	0

Réduction : 0

	N	P	L
P	0	+∞	0
L	164	0	+∞
S	+∞	0	0

Appel récursif n°3 – droite



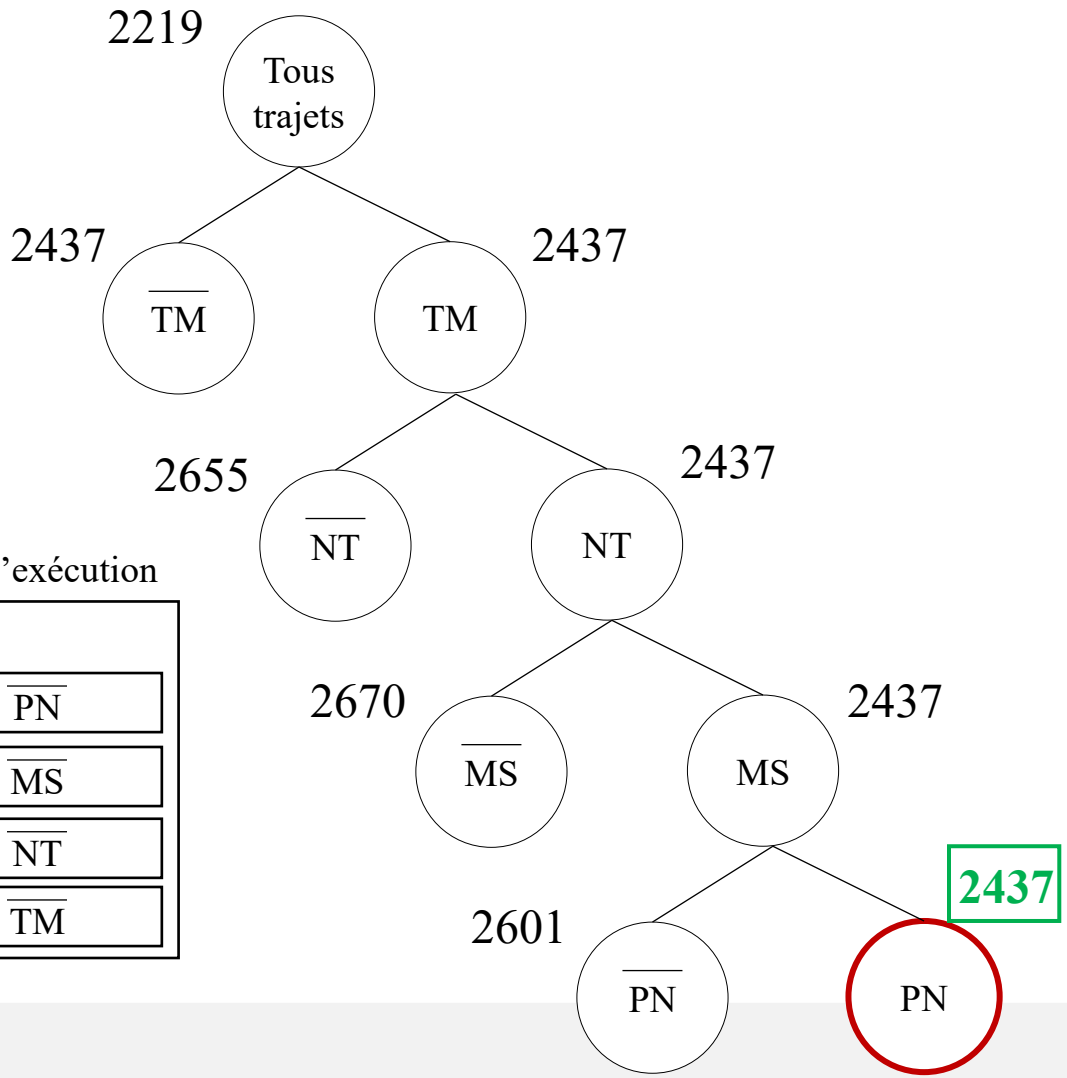
	N	P	L
P	0	+∞	0
L	164	0	+∞
S	+∞	0	0

Réduction : 0

	N	P	L
P	0 **(164)**	+∞	0 **(0)**
L	164	0 **(164)**	+∞
S	+∞	0 **(0)**	0 **(0)**

Regret maximal : Paris-Nantes (choix arbitraire)

Appel récursif n°4 – droite



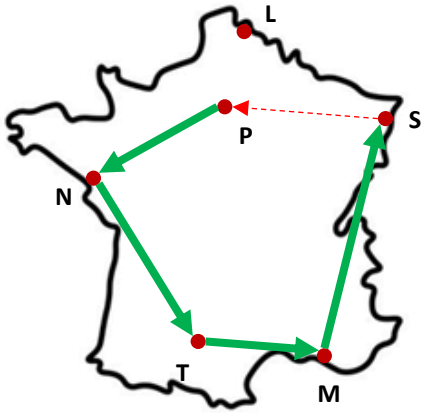
Pile d'exécution

$\overline{PN}$
$\overline{MS}$
$\overline{NT}$
$\overline{TM}$

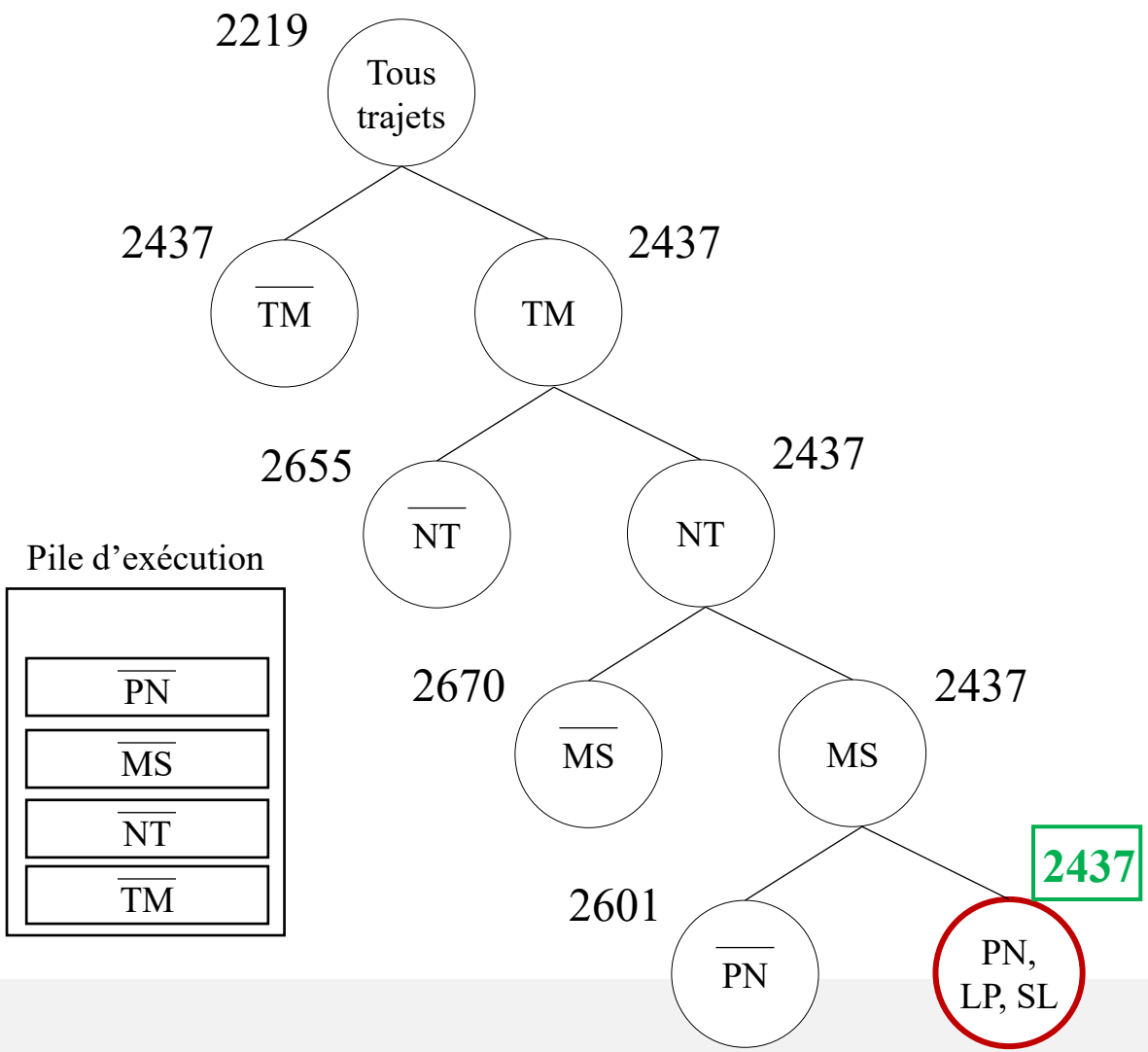
	P	L
L	0	+∞
S	+∞	0

Trajet parasite éliminé : SP

⇒ sous-tournée PN-NT-TM-MS-SP



Appel récursif n°4 – droite



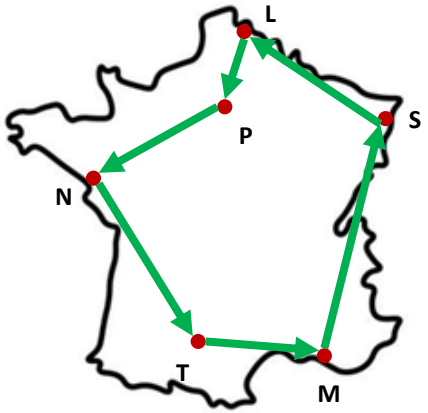
	P	L
L	0	+∞
S	+∞	0

Trajet parasite éliminé : SP

⇒ sous-tournée PN-NT-TM-MS-SP

Matrice 2x2 : il n'y a plus qu'une seule manière de terminer la tournée

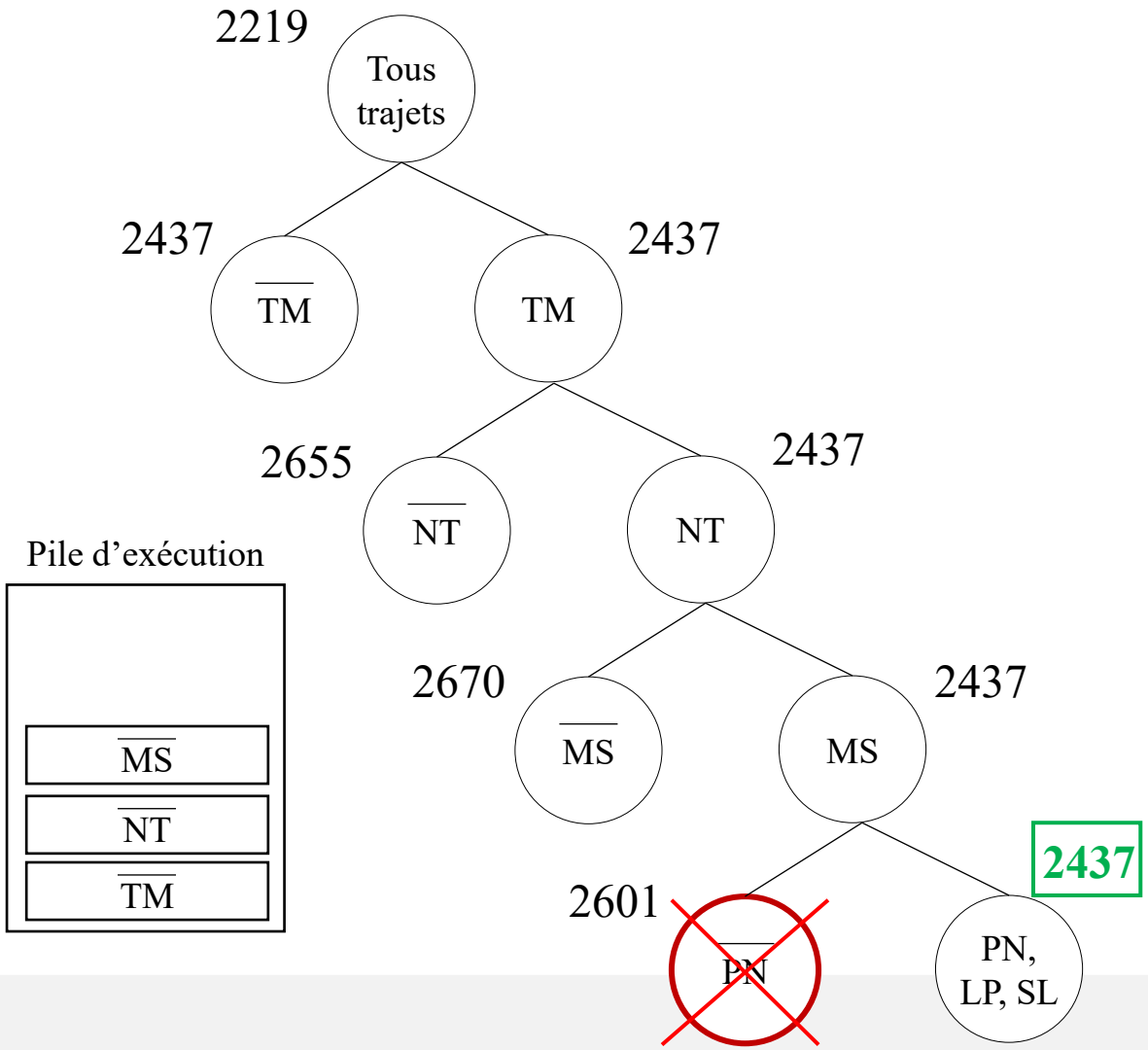
Ici, inclure LP et SL



⇒ Tournée complète

Coût : **2437**

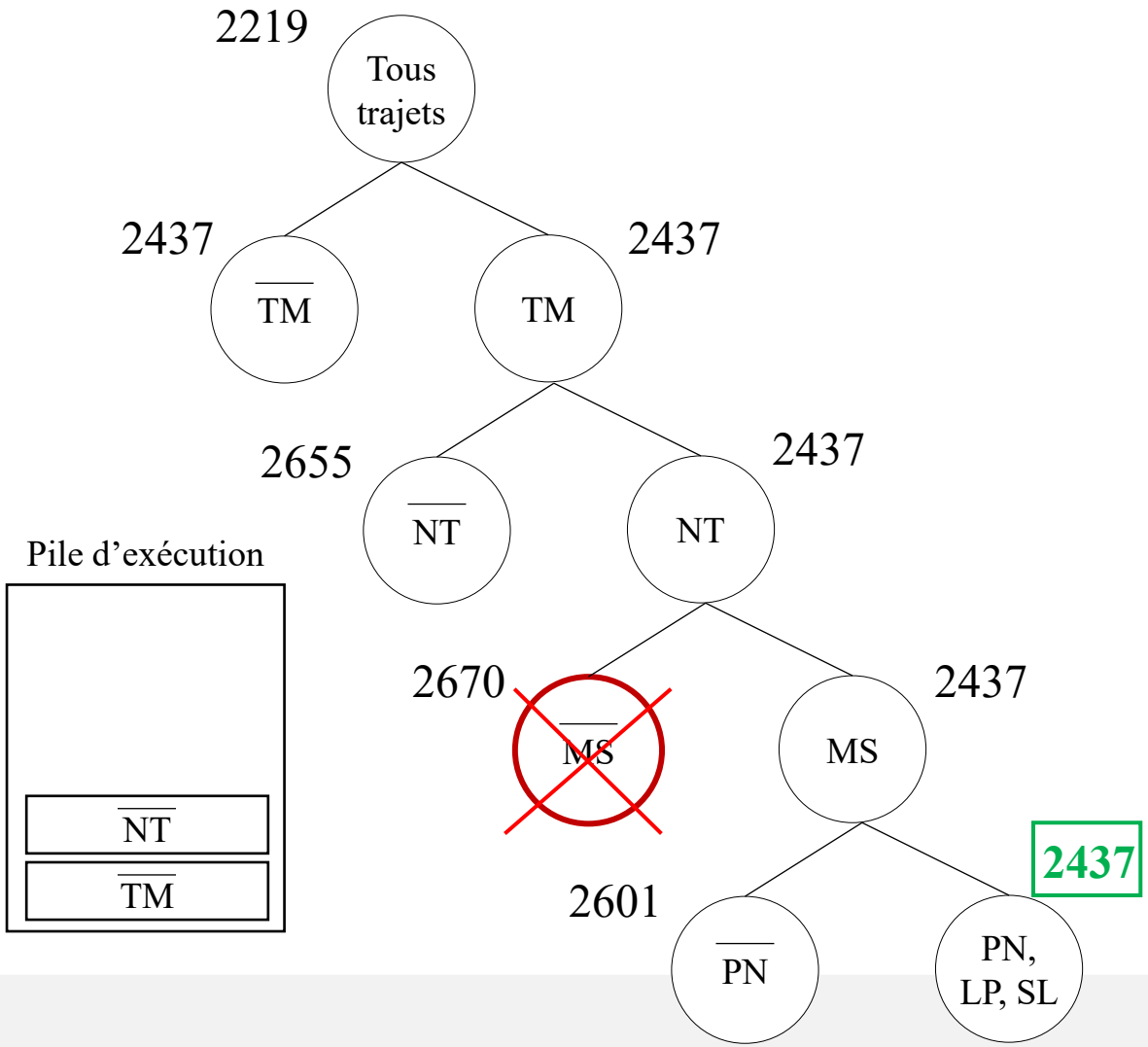
Appel récursif n°4 – gauche



Borne inférieure 2610 ≥ 2437

⇒ Coupe

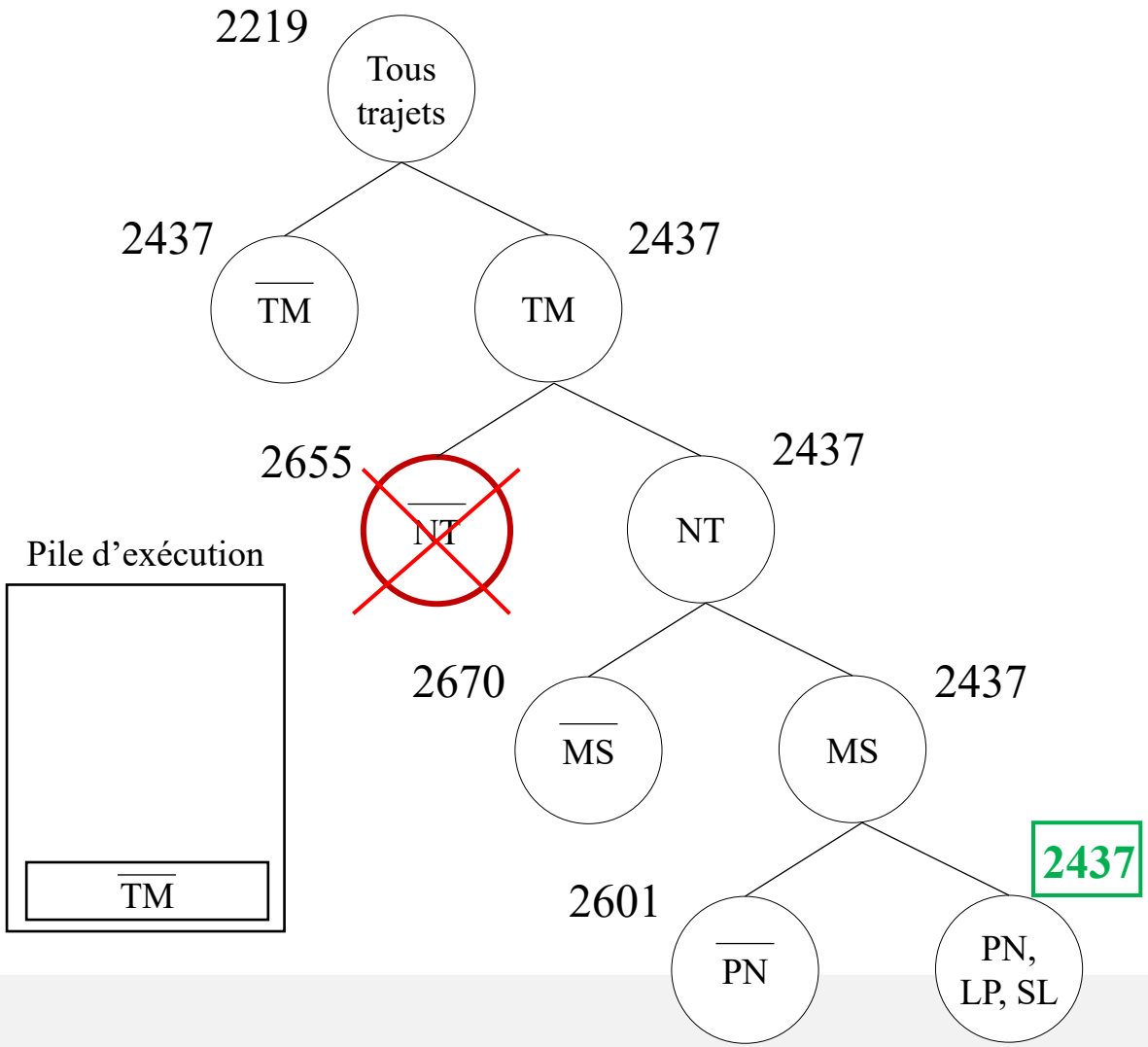
Appel récursif n°3 – gauche



Borne inférieure 2670 ≥ 2437

⇒ Coupe

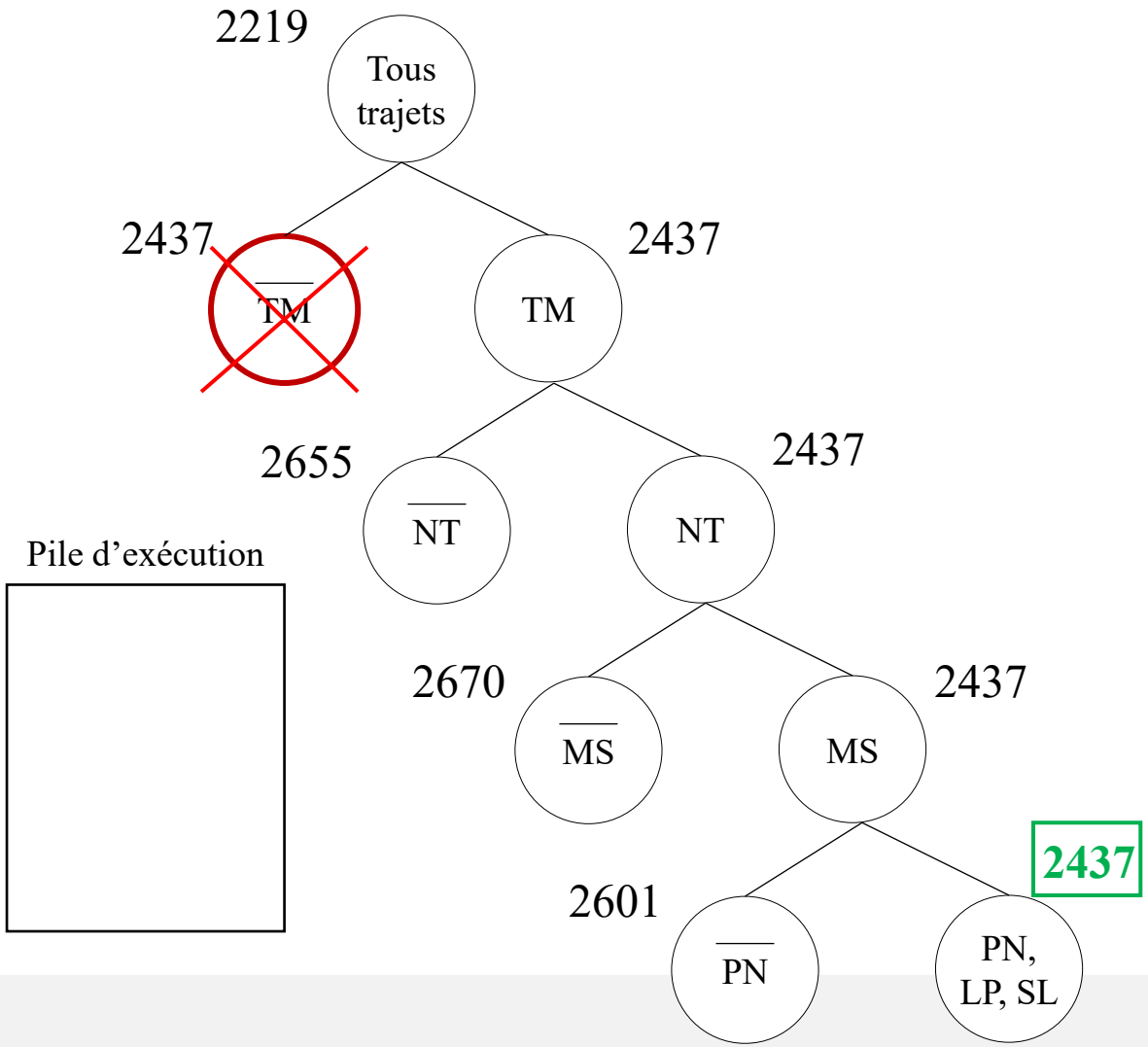
Appel récursif n°2 – gauche



Borne inférieure 2655 ≥ 2437

⇒ Coupe

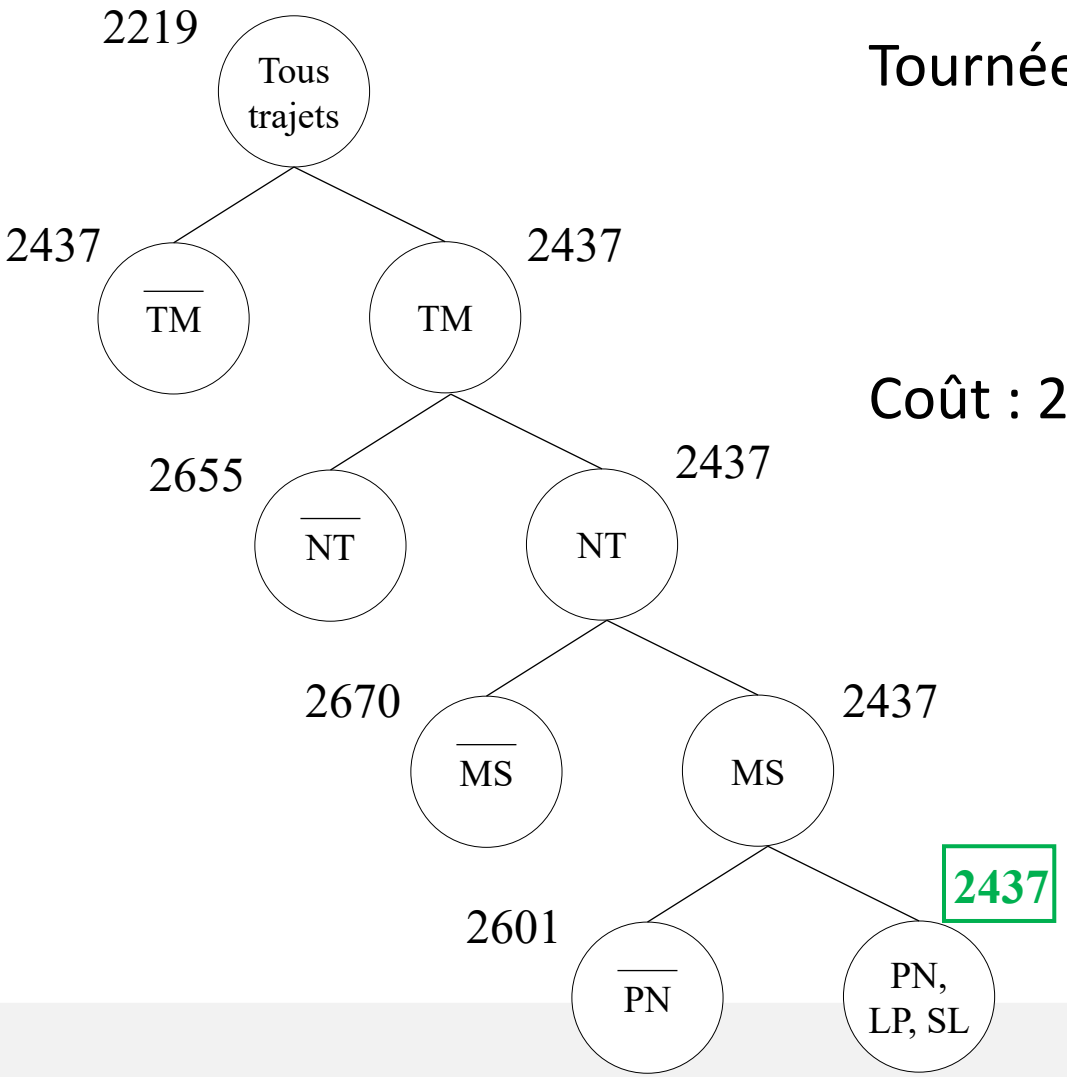
Appel récursif n°1 – gauche



Borne inférieure 2437 ≥ 2437

⇒ Coupe

Fin des appels récursifs

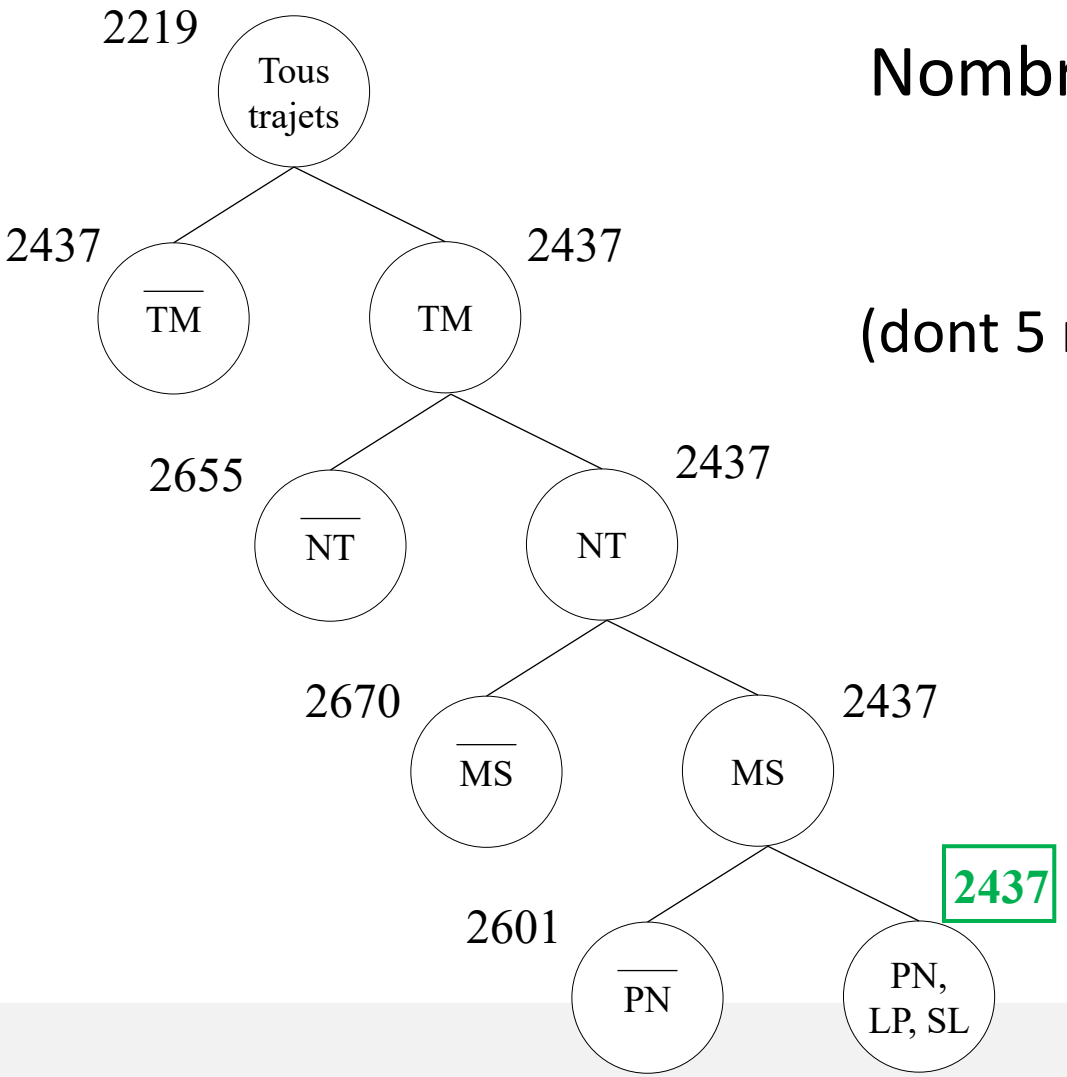


Tournée optimale au départ de Toulouse :

TM-MS-SL-LP-PN-NT

Coût : 2437 km

Efficacité de l'élagage



Nombre de nœuds : 9 << 1957

Force brute

(dont 5 réellement visités)